

Enterprise AI -- Platform Walkthrough

Legacy Software Intelligence & Modernization Platform | Complete Guide (Phases P0 - P22)

Implementation: Phases P0 - P22 Complete	Backend Tests: 309 / 309 Passing (Pytest)	Frontend Tests: 6 / 6 Passing (Vitest)
Cost Mode: 100% Deterministic (Zero Paid APIs)	Evaluation: 35 Grounded Items (0% Hallucination)	Target Stack: Java, Spring Boot, JPA, PostgreSQL

EXECUTIVE BRIEFING & SYSTEM PURPOSE

Enterprise AI is a production-ready legacy software intelligence platform designed specifically for Java and Spring Boot enterprise ecosystems. It provides automated codebase discovery, dependency graph extraction, end-to-end request flow tracing, change-impact blast radius computation, deterministic risk detection, and modernization planning -- operating entirely locally with zero required paid APIs or cloud dependencies. Every insight is strictly grounded in AST syntax and relational persistence models.

CORE ARCHITECTURAL INVARIANTS

- 1. Zero Code Invention:** The system never invents classes, methods, endpoints, or dependencies.
- 2. Deterministic Grounding:** AST parser and SQL persistence are the single source of truth.
- 3. Evidence-First Synthesis:** Answers must cite exact source files, line numbers, and relationships.
- 4. Complete Privacy:** Source code is never transmitted to external third-party LLM providers.

1. End-to-End Architectural Pipeline

Stage	Module	Technical Responsibility & Guarantees	Determinism
1. Ingestion	Safe Scanner / ZIP Reader	Directory & ZIP scanning. Blocks path traversal attacks (Zip-Slip) using canonical path validation. Discovers Java, SQL, config, and docs.	100% Deterministic
2. AST Parsing	Java AST & Annotation Engine	Extracts packages, classes, interfaces, methods, constructors, fields, Spring REST endpoints (@GetMapping, @PostMapping), and database entities.	100% Deterministic
3. Graph Synth	Dependency Graph Extractor	Builds typed directed graph across 11 edge types (CALLS, IMPORTS, EXTENDS, QUERIES, EXPOSES, etc.). Preserves and reports unresolved calls.	100% Deterministic
4. Persistence	PostgreSQL / Alembic Store	Stores immutable historical snapshots across 21 relational tables. Supports historical diffs, symbol searches, and graph queries.	100% Deterministic
5. Graph Engine	Knowledge Graph Traversal	Executes bounded BFS/DFS traversals, shortest path calculations, request flow tracing, and Controller -> Service -> Repo architecture validation.	100% Deterministic
6. Retrieval	Structured & Semantic Layer	Structured symbol lookup + local embeddings fallback. Packages exact file:line source evidence cards for human and machine inspection.	100% Deterministic
7. Presentation	CLI, FastAPI & React App	13-command CLI, 15+ REST endpoints, and a 10-page React dashboard delivering interactive flow graphs, blast radius trees, and executive reports.	Zero Hallucination

2. Complete Implementation Walkthrough (Phases P0 - P22)

All 23 planned phases have been engineered, integrated, and validated with automated regression suites.

A. Ingestion, AST Parsing & Knowledge Graph (Phases P0 - P5)

- **P0: Setup & Health Scaffolding:** Configured Python 3.11 backend with FastAPI, Pydantic, SQLAlchemy, Alembic migrations, and verified GET /health with CORS.
- **P1: Repository Scanner & Zip-Slip Defense:** Directory and safe ZIP ingestion. Categorizes Java, Config, SQL, and Docs. Validates against directory traversal (Zip-Slip) attacks.
- **P2: Java AST & Annotation Extractor:** AST parser capturing classes, interfaces, methods, constructors, fields, Spring REST endpoints (@GetMapping, @PostMapping), and JPA tables.
- **P3: Dependency Graph Model:** Typed directed graph across 11 relationships: CONTAINS, IMPORTS, EXTENDS, IMPLEMENTS, CALLS, USES, EXPOSES, QUERIES, DEPENDS_ON, REFERENCES, TESTS.
- **P4: PostgreSQL Persistence & Snapshots:** Relational persistence across 21 domain tables. Stores complete analysis snapshots, indexes graph nodes/edges, and tracks historical runs.
- **P5: Knowledge Graph Traversal Engine:** Graph traversal supporting direct neighbors, upstream callers, downstream callees, bounded impact traversal, and verified 4-tier architectural chains.

B. CLI, Retrieval Pipeline & Evidence Synthesis (Phases P6 - P10)

- **P6: Unified Analysis CLI:** Comprehensive command-line tool with 13 subcommands, supporting dual-syntax invocation (analyze.py or analyze.py).
- **P7: Structured Symbol Retrieval:** Extracts exact symbols, callers, callees, and architectural chains using symbol tables and graph traversal without requiring AI models.
- **P8: Local Semantic Retrieval Fallback:** Optional local embedding engine for documentation and natural language queries, featuring automatic deterministic keyword fallback.
- **P9: Local LLM Provider Abstraction:** Pluggable LLM interface supporting local models (Ollama/llama3) while defaulting to zero-cost deterministic analysis if no local model is running.
- **P10: Evidence-Grounded Q&A; Engine:** Synthesizes precise answers citing verified file:line references, request flows, and explicit disclosures of unresolved calls and limitations.

C. Core Software Intelligence & Reasoning Engines (Phases P11 - P16)

- **P11: Request Flow Tracing:** Traces full execution paths from HTTP endpoints through controllers, service orchestrators, and repositories down to physical SQL tables.
- **P12: Change-Impact Blast Radius:** Computes upstream callers, downstream dependencies, affected APIs, database entities, and associated test suites when a symbol changes.
- **P13: Measurable Technical Risk Engine:** Audits structural health: circular dependencies, high fan-out, high fan-in, unlayered database coupling, missing tests, and God classes.
- **P14: Architecture Visualization Data:** Generates layered architectural topology data categorizing nodes by architectural role for visual inspection in frontend graph viewers.
- **P15: Evidence-Backed Modernization Advisor:** Detects monolithic coupling, circular dependencies, and legacy framework patterns, ranking refactoring targets by structural dependency.
- **P16: Comprehensive Multi-Format Reporting:** Generates formal 11-section reports in Markdown and JSON covering executive summaries, inventory, architecture, risk matrices, and roadmaps.

D. Web Application, Security & Evaluation (Phases P17 - P22)

- **P17: Production FastAPI Endpoints:** Comprehensive REST API exposing project lifecycle, file/class browsers, graph traversal, request tracing, impact analysis, and report generation.
- **P18: Functional React Dashboard (10 Pages):** Full single-page application built with React, Vite, and TypeScript featuring 10 dedicated views with real-time metrics and graph visualizers.
- **P19: Security Foundations:** Authentication abstraction, project tenant isolation, upload size controls, secret sanitization (redacting tokens/keys), and audit logging.
- **P20: Multi-Service Docker Orchestration:** Multi-stage Dockerfiles and docker-compose orchestration coordinating PostgreSQL 16, FastAPI backend, and React/Nginx frontend.
- **P21: 35-Item Grounded Evaluation Suite:** Formal benchmark covering 35 complex questions across 10 categories, evaluating precision, recall, and verifying 0% hallucination.
- **P22: Automated Test & Validation Harness:** Continuous verification suite running 309 pytest backend tests and 6 vitest frontend tests with 100% pass rate.

3. Unified CLI Command Reference (13 Subcommands)

The CLI provides both high-level project summaries and granular symbol-level inspection. It supports dual invocation: `python analyzer-cli/analyze.py <path> <command>` or `python analyzer-cli/analyze.py <command> <path>`.

Command	Key Arguments & Flags	Output & Technical Functionality
analyze	--persist, --json, --list-files	Default command. Scans repo, parses Java AST, extracts dependencies, validates architecture, and audits risks.
inspect	(none)	Inventory summary, package hierarchy, database references, and verified architectural chains.
list-classes	(none)	Enumerates all discovered classes, interfaces, records, and enums with line numbers and source file paths.
list-methods	(none)	Lists all methods with return types, signatures, annotations, parameter lists, and enclosing classes.
show-class	<ClassName>	Detailed breakdown of class annotations, fields, methods, implemented interfaces, and direct dependencies.
show-dependencies	[symbol]	Displays incoming callers, outgoing callees, and overall relationship type distribution.
show-graph	--format dot json	Outputs graph topology metrics, node kind distribution, and optional Graphviz DOT/JSON representations.
trace-request	</api/path>	Traces sequential request flow: Endpoint -> Controller -> Service -> Repository -> Table with verification status.
impact	<Symbol> [--depth N]	Calculates change blast radius: directly/indirectly affected classes, APIs, databases, and test suites.
risks	(none)	Outputs deterministic risk findings: circular dependencies, high fan-out, missing tests, coupling hotspots.
ask	"<Question>"	Evidence-backed natural language Q&A; synthesizing verified code traces and explicit analysis limitations.
report	--format md json --output <f>	Generates an 11-section executive intelligence report in Markdown or JSON format.
architecture	(none)	Displays layered architecture breakdown: Controllers, Services, Repositories, Entities, and SQL Tables.

CLI RUNTIME FLAGS & EXTENSIONS

- Common Operational CLI Flags:**
- persist: Persists analysis results as an immutable snapshot in PostgreSQL.
 - json: Emits machine-readable JSON output for CI/CD integration.
 - scan-only: Performs file discovery and inventory categorization without AST parsing.
 - project-name <name>: Custom project label assigned to the persisted run snapshot.
 - list-files / --list-types: Prints exhaustive file catalog or extracted type hierarchy.

4. Live Case Study: payment-service Fixture

The platform includes a real-world Spring Boot microservice fixture (sample-projects/payment-service) engineered with layered REST services, JPA persistence, intentional circular dependencies, high fan-out components, and partial test coverage to validate real-world detection capabilities.

A. Request Flow Tracing Execution

Executing trace-request /api/payment traces the complete path down to the database table:

```
$ python analyzer-cli/analyze.py ./sample-projects/payment-service trace-request /api/payment

Request Flow for: /api/payment
1. [endpoint]      POST /api/payment (src/.../controller/PaymentController.java:27)
2. [method]        create --[EXPOSES]--> (PaymentController.java)
3. [method]        processPayment --[CALLS]--> (PaymentService.java)
4. [method]        save --[CALLS]--> (PaymentRepository.java)
5. [database_table] payments --[QUERIES]--> (PaymentEntity.java)
[OK] Reaches database table: payments
Unresolved calls: String.valueOf, body.get, ResponseEntity.ok, entity.setAmount
```

B. Deterministic Risk Findings Audit (16 Total Findings)

Running risks detected 16 measurable findings across the codebase without arbitrary AI scoring:

Severity	Category	Component	Measurable Evidence & Architectural Impact
HIGH	CIRCULAR_DEPENDENCY	NotificationService <-> PaymentService	Bidirectional call cycle detected: NotificationService calls PaymentService, and PaymentService calls NotificationService.
HIGH	HIGH_COUPLING	PaymentService	High bidirectional coupling: 8 incoming dependencies, 7 outgoing dependencies, 4 callers, 4 callees.
HIGH	HIGH_DEPENDENCY_COMPONENT	MetricsFacade	Central structural bottleneck: 6 incoming callers with zero downstream callees.
MEDIUM	HIGH_FAN_OUT	RefundController	Controller has high fan-out coupling (5 outgoing dependencies).
MEDIUM	TESTING_GAP	NotificationService	Zero automated tests reference or exercise NotificationService.
MEDIUM	DATABASE_COUPLING	PaymentRepository	Directly binds to PaymentEntity and physical SQL table 'payments'.

C. Change Impact Blast Radius Execution

```
$ python analyzer-cli/analyze.py ./sample-projects/payment-service impact PaymentService.processPayment

Impact Analysis for: processPayment (method)
Directly Affected: create (PaymentController.java), processPaymentPersistsRow (PaymentServiceTest.java)
Affected Database Objects: payments (table), PaymentEntity (class)
Affected Tests: processPaymentPersistsRow (PaymentServiceTest.java)
Note: Static impact analysis identifies potentially affected entities based on compile-time call graphs.
```

5. Interactive Web Dashboard (10 Dedicated Views)

The React 18 + Vite + TypeScript frontend provides 10 dedicated navigation views for engineering teams:

View	Interface Functionality & Capabilities
Projects & Upload	Manage active projects, upload fresh repositories or ZIP archives, and trigger background analyses.
Overview Dashboard	Executive summary with key metric cards: Total Files, Java Classes, Methods, Dependencies, Endpoints, and Tables.
Architecture Explorer	Interactive multi-tier chain visualizer displaying verified Controller -> Service -> Repository -> Table flows.
Codebase Browser	File tree and symbol catalog with instant filtering across classes, methods, annotations, and source lines.
Dependency Matrix	Graph topological statistics, relationship breakdown by type (CALLS, IMPORTS), and upstream/downstream callers.
Ask Codebase	Evidence-grounded conversational terminal. Returns verified text answers, step flows, and exact file:line cards.
Impact Analyzer	Blast-radius simulator: select any class/method to inspect directly/indirectly impacted callers, APIs, and tests.
Risk Assessment	Structural audit categorizing findings by severity (HIGH/MEDIUM) with exact metrics (fan-in/out, circularity).
Modernization Planner	Refactoring matrix identifying monolith decomposition candidates, high coupling bottlenecks, and framework upgrades.
Exportable Reports	One-click generation and download of complete executive intelligence reports in Markdown or JSON.

6. Quality Assurance, Benchmark & Quick Start

BENCHMARK METRICS & TEST VERIFICATION

Evaluation Benchmark Suite Results (P21):

- **Scope:** 35 Grounded Evaluation Items across 10 categories (Architecture, Request Flow, Dependencies, Classes, Methods, APIs, Database, Impact, Risk, Modernization).
- **Hallucination Rate: 0.0%** (100% of entities, calls, and relationships verified against AST ground truth).
- **Automated Test Suite:** 309 backend pytest tests (11.36s execution) + 6 frontend vitest tests passing (100% pass rate).

```
# 1. Analyze sample project via CLI (Zero external configuration required)
python analyzer-cli/analyze.py ./sample-projects/payment-service

# 2. Trace request flows and change impact
python analyzer-cli/analyze.py ./sample-projects/payment-service trace-request /api/payment
python analyzer-cli/analyze.py ./sample-projects/payment-service impact PaymentService.processPayment

# 3. Launch full stack with Docker Compose (PostgreSQL 16 + FastAPI Backend + React Dashboard)
docker compose up -d --build
# Dashboard: http://localhost:3000 | FastAPI Swagger API: http://localhost:8000/docs
```

VERIFICATION STATUS: ALL 23 PHASES VALIDATED & APPROVED

Enterprise AI delivers a rigorous, self-contained, enterprise-grade software intelligence solution. By enforcing strict AST determinism, relational persistence, and zero paid API dependencies, organizations can analyze, govern, and modernize legacy Java systems with complete architectural fidelity.